Introducción JFLAP

- JFLAP es un software para experimentar con temas de lenguajes formales que incluyen autómatas finitos no deterministas, autómatas pushdown no deterministas, máquinas Turing multicinta, varios tipos de gramáticas, análisis y sistemas L. Además de construir y probar ejemplos para estos, JFLAP le permite a uno experimentar con pruebas de construcción de una forma a otra, como convertir un NFA a un DFA a un DFA de estado mínimo a una expresión regular o gramática regular. Haga clic aquí para obtener más información sobre lo que se puede hacer con JFLAP.

Historia de JFLAP

- JFLAP comenzó como una serie de herramientas en el Instituto Politécnico Rensselaer alrededor de 1990, con estudiantes trabajando bajo la dirección de Susan Rodger. El proyecto JFLAP se mudó a la Universidad de Duke en 1994 cuando Rodger se mudó allí.

A continuación, se incluye una lista de estudiantes que trabajaron en JFLAP y / o herramientas relacionadas a lo largo de los años.

DATE	NAME	SCHOOL	TOOL
1990	Dan Caugherty	Rensselaer	NPDA
1991	Grant Poladian	Rensselaer	Tools for Automata
1992	Mark LoSacco	Rensselaer	FLAP
1992	Mike James	Rensselaer	LR Parser
1992	Danny Daglas	Rensselaer	Turing Machine Implementation
1992	Jeffrey Nesheiwat	Rensselaer	Turing Machine Implementation
1992	Jasper Wong	Rensselaer	Turing Machine Implementation
1993	Eric Luce	Rensselaer	Turing Machine Building Blocks
1993	Steve Blythe	Rensselaer	LR Parser
1994	Ugur Dogrusoz	Rensselaer	Automatic Layout of DFA in LR Parsing
1994	Bhasker Vasudevan	Rensselaer	NFA to DFA
1994	Mohan Nibhunupudi	Rensselaer	CFG to CNF
1995	Edwin Tsang	Duke University	LL and LR Parse Trees
1995	Greg Badros	Duke University	Modifications to FLAP
1995	Steve Wolfman	Duke University	L-systems
1995	Ben Hardekopf	Duke University	DFA to minimum DFA
1996	Ken Leider	Duke University	PumpLemma
1996	Anna Bilska	Duke University	Pate: Brute Force Parser
1996	Jason Salemme	Duke University	Pate: Brute Force Parser
1996	Magda Procopiuc	Duke University	JFLAP - FLAP to Java
1996	Octavian Procopiuc	Duke University	JFLAP - FLAP to Java

1997	Alex Karweit	Duke University	JeLLRap - LL and LR parsing
1997	Robyn Geer	Duke University	JeLLRap - LL and LR parsing
1997	Eric Gramond	Duke University	Extensions to JFLAP
1998	Lenore Ramm	Duke University	L-systems
1999	Ted Hung	Duke University	Regular Expressions
2002	Thomas Finley	Duke University	JFLAP rewritten and extended in Swing, L-Systems, Many new additions
2002	Ryan Cavalcante	Duke University	JFLAP rewritten and extended in Swing
2005	Stephen Reading	Duke University	Turing Machine Building Blocks in JFLAP, Batch processing
2005	Bart Bressler	Duke University	Turing Machine Building Blocks in JFLAP
2006	Jinghui Lim	Duke University	Moore and Mealy machines, Pumping Lemma
2007	Chris Morgan	Duke University	Graph Layouts, JFLAP Tutorial, Pumping Lemma
2007	Kyung Min (Jason) Lee	Duke University	CYK Parser, User-Control Parser, JFLAP Tutorial, TM to UG
2008	Jonathan Su	Duke University	Usage and Updating of JFLAP, image formats, flexible transitions
2009	Henry Qin	Duke University	Rewrite of Turing machine Building Blocks, Many other updates.
2011	Julian Genkins	Duke University	Started redesign of JFLAP to JFLAP 8.0
2012	Peggy Li	Duke University	experimenting with sets
2012	Ian McMahon	Duke University	CYK parsing, more on Turing machines, more on redesign of JFLAP
2014	Aohui (Lawrence) Lin	Duke University	Parse Tree, Regex to NFA conversion and more redesign of JFLAP
2014	John Godbey	Duke University	More redesign of JFLAP
2015	James Cho	Duke University	Experiment with JFLAP to HTML5 with OpenDSA
2015	Martin Tamayo	Duke University	JFLAP to HTML5 with OpenDSA

2015	Sung-Hoon Kim	Duke University	JFLAP to HTML5 with OpenDSA
2016	Han "Bill" Yu	Duke University	JFLAP to HTML5 with OpenDSA
2016	Brian Pester	Duke University	JFLAP to HTML5 with OpenDSA
2017	Wei-Ting Yeh	Duke University	JFLAP to HTML5 with OpenDSA
2017	Yanbo Fang	Duke University	JFLAP to HTML5 with OpenDSA
2018	Jay Patel	Duke University	Update JFLAP to 7.1, DFA to RE and RG in OpenDSA

Objetivos

Objetivo General:

- Estudiar el uso de herramientas graficas que ofrece JFLAP para la creación de autómatas finitos.

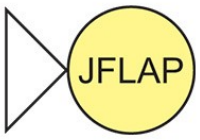
Objetivos Específicos:

- Conocer las posibilidades de JFLAP como herramienta de demostración de autómatas.
- Familiarización con los autómatas empleando JFLAP.
- Generación del autómata finito equivalente a la expresión regular y gramáticas implementadas.
- Integración con la herramienta JFLAP para la resolución de ejercicios.

Marco teórico

JFLAP es una herramienta Java que facilita la experimentación con autómatas finitos, autómatas a pila y máquinas de Turing, y su equivalencia con expresiones regulares y gramáticas. JFLAP permite la definición de una máquina de estados finitos y permite al usuario experimentar con diferentes cadenas de entrada.

Estas funcionalidades hacen de JFLAP una herramienta idónea para el apoyo a asignaturas de Autómatas, gramáticas y lenguajes formales, comunes en los planes de estudio de las titulaciones de Informática, en este documento nos enfocaremos principalmente en la creación de autómatas finitos.



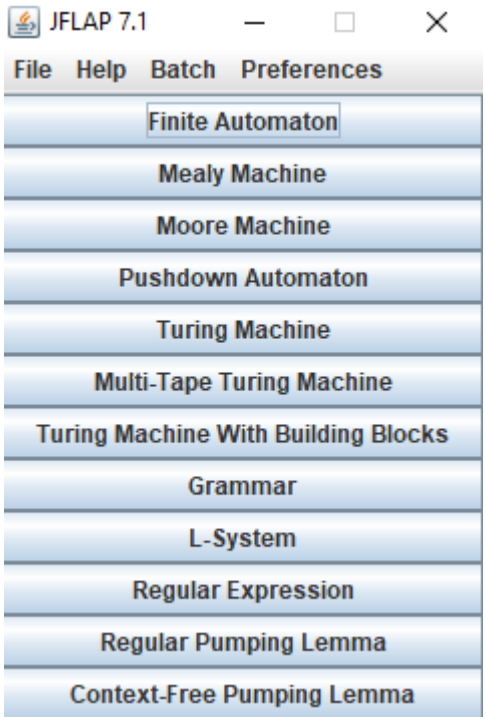
Versión utilizada:
JFLAP Version 7.1
RELEASED July 27, 2018

Creación de Autómata paso a paso:

Ejercicio:

$$L(DFA_2)=L(M_2)=\left[\frac{w}{w}\in\Sigma^{¿},\Sigma=[0,1],w'1,w'\in\Sigma^{¿}\right]$$

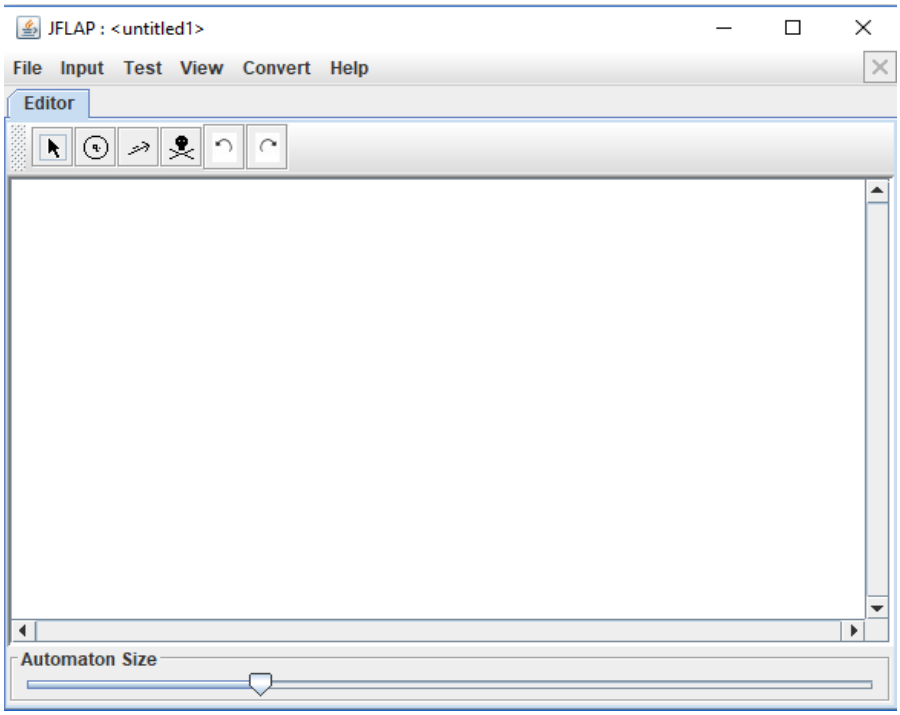
1. Abrimos JFLAP:



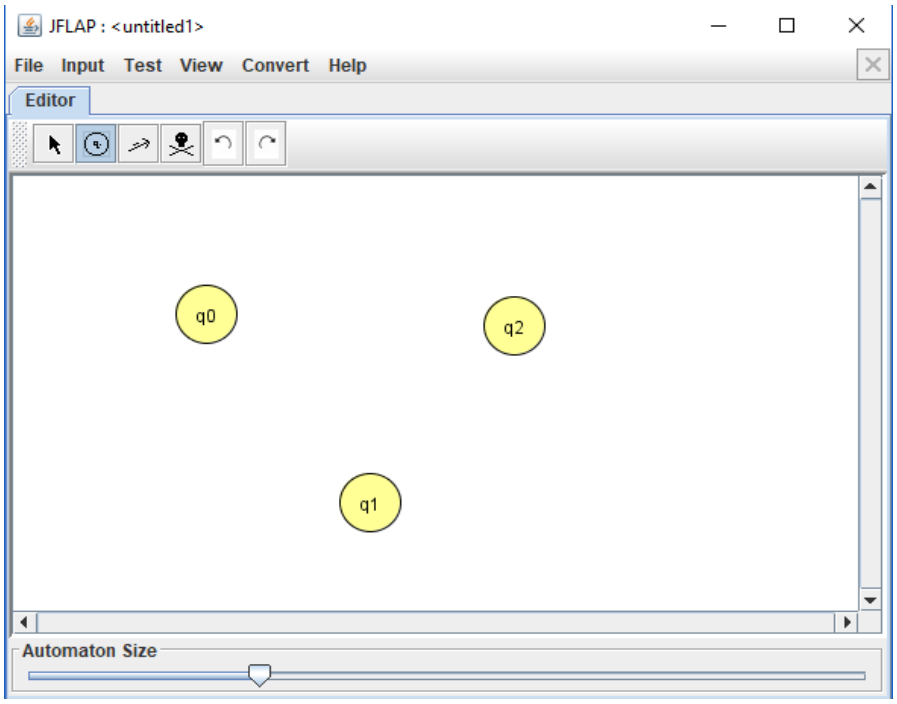
Al ejecutar este software aparece el menú que aparece arriba. En el que se puede apreciar todas las opciones que dispone este programa y que abarcan la gran mayoría de los temas que trata la informática teórica o teoría de la computación:

- Autómatas Finitos (AF)
- Maquinas Secuenciales (*Mealy y Moore*)
- Autómatas de Pila (*Pushdown*)
- Máquinas de Turing
- Gramáticas para estructuras de frases
- Sistemas-L (de *Lindenmayer*)

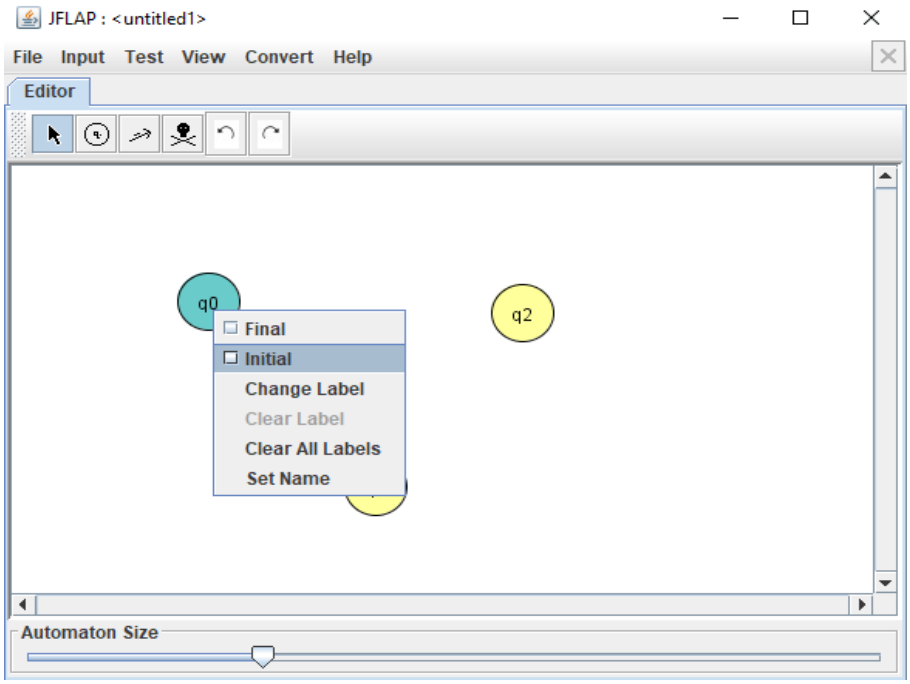
2. En nuestro caso seleccionamos la opción *Finite Automaton*:



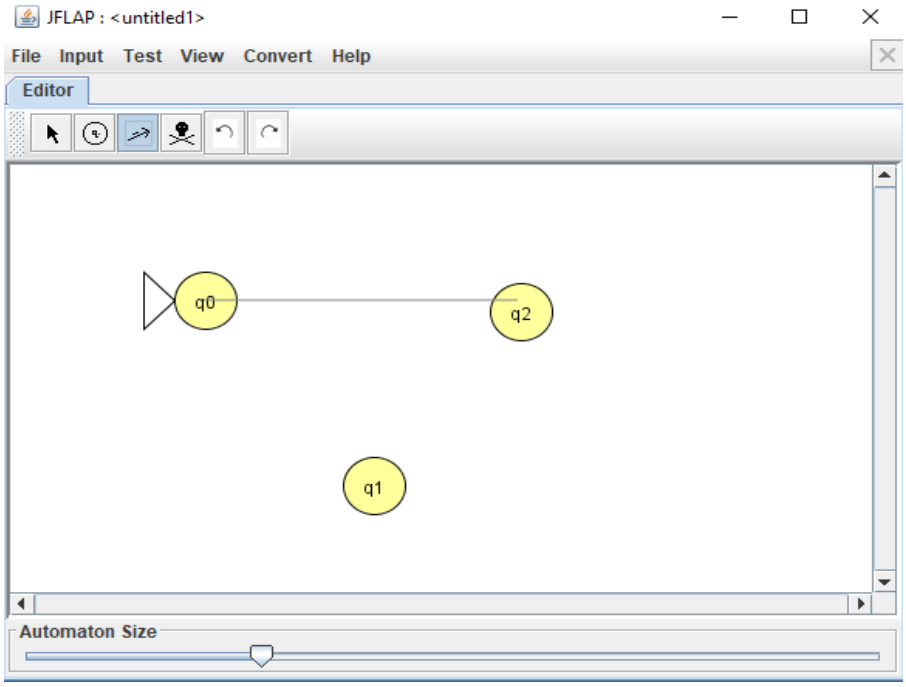
3. Empezamos con la creación de nuestros estados que se realiza con la 2 opción de izquierda a derecha del editor:



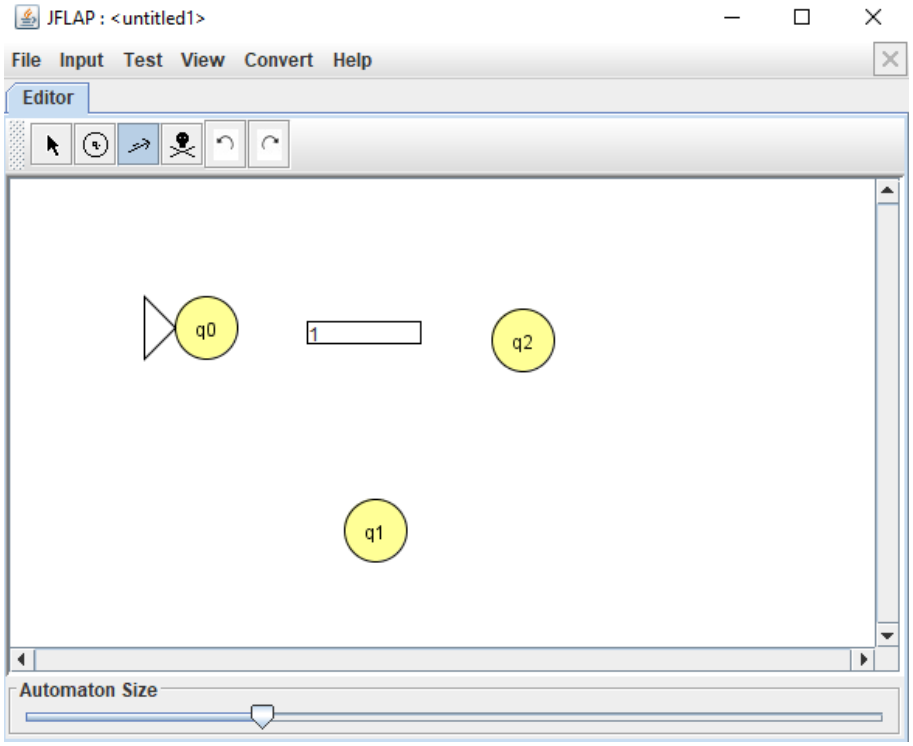
4. Luego seleccionamos la 1 opción de izquierda a derecha del editor para indicar al software nuestro estado inicial:

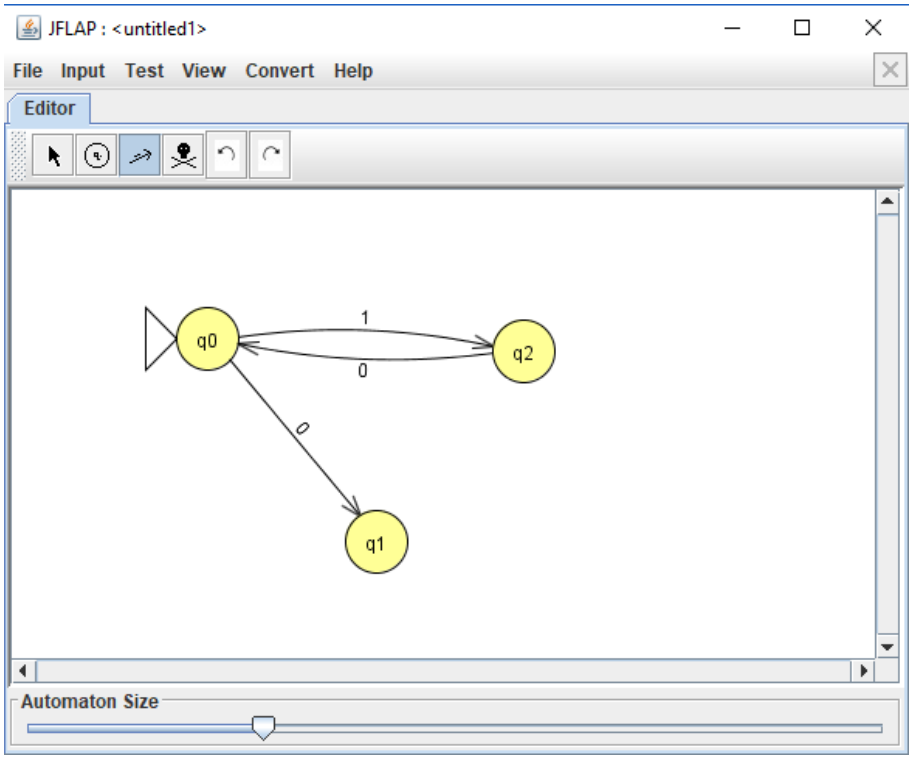


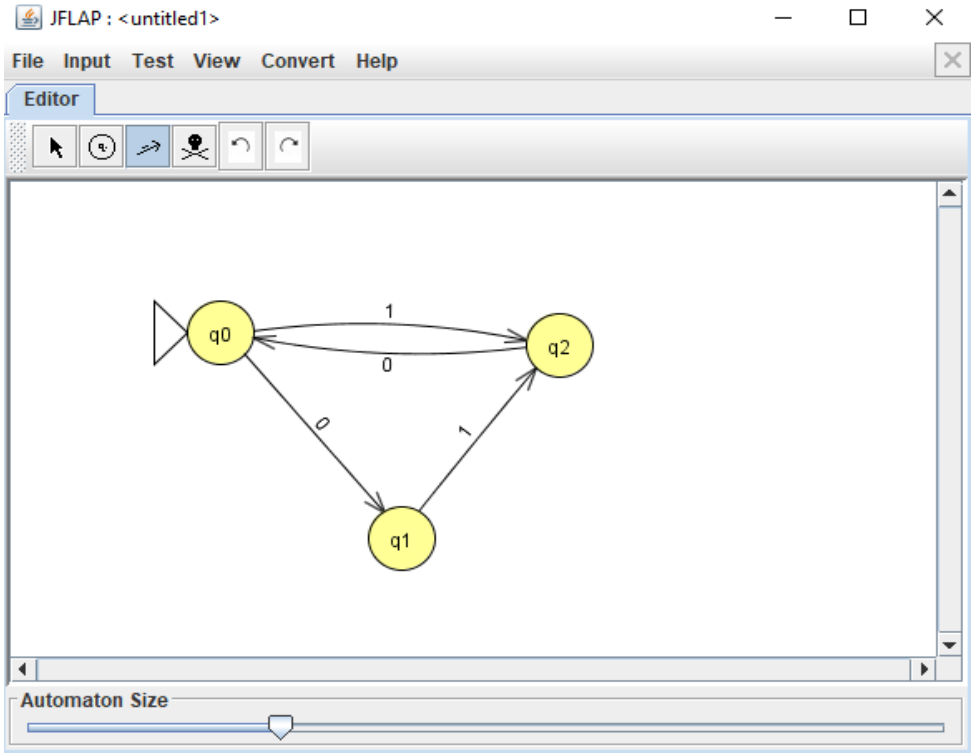
5. Luego seleccionamos la 3 opción de izquierda a derecha del editor para crear nuestras transiciones que en este caso se basaran por medio del alfabeto propuesto por el ejercicio el cual es $\Sigma=[0,1]$



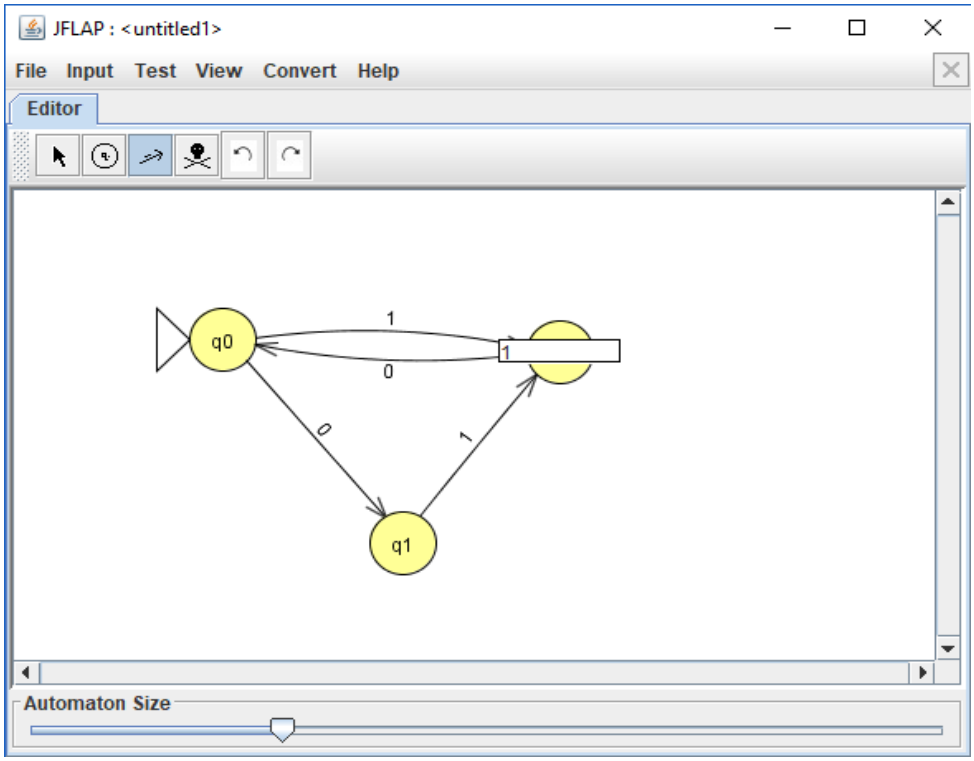
6. Introducimos el valor que llevara esa transición y realizamos las siguientes transiciones para dar solución ha dicho ejercicio:



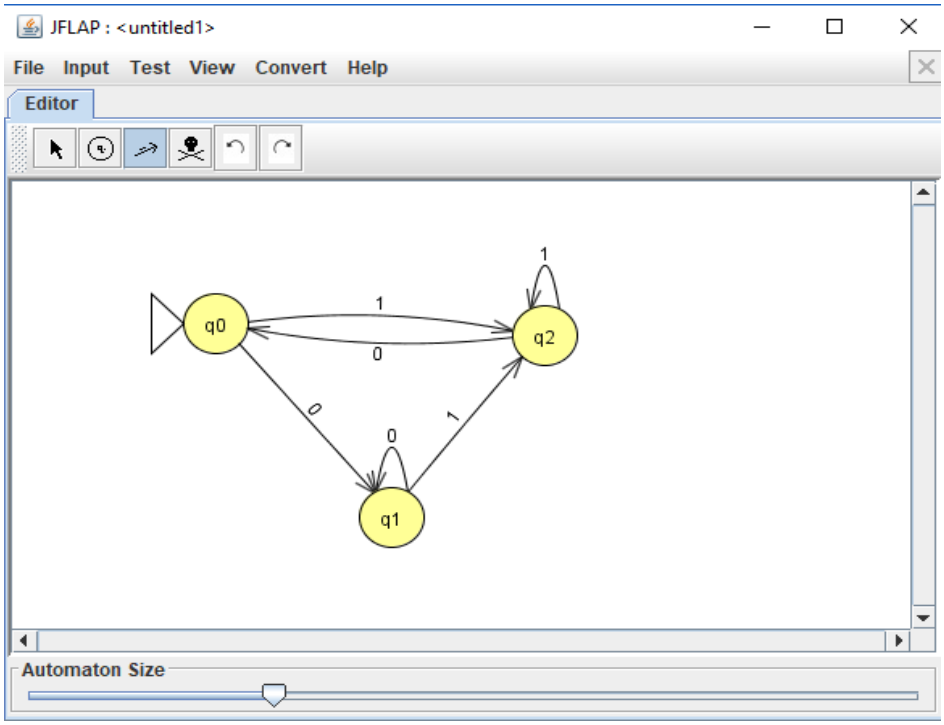




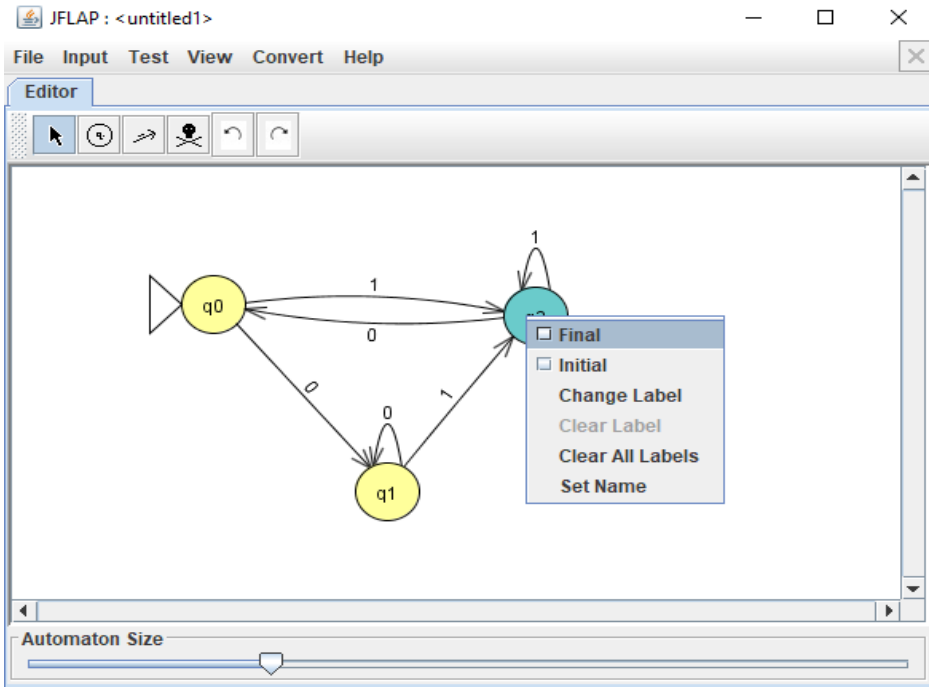
7. Llegado a este punto necesitamos una transición recursiva en el estado q_2 que se realiza con la opción de transición dando clic al estado que se desea aplicar dicha recursividad:



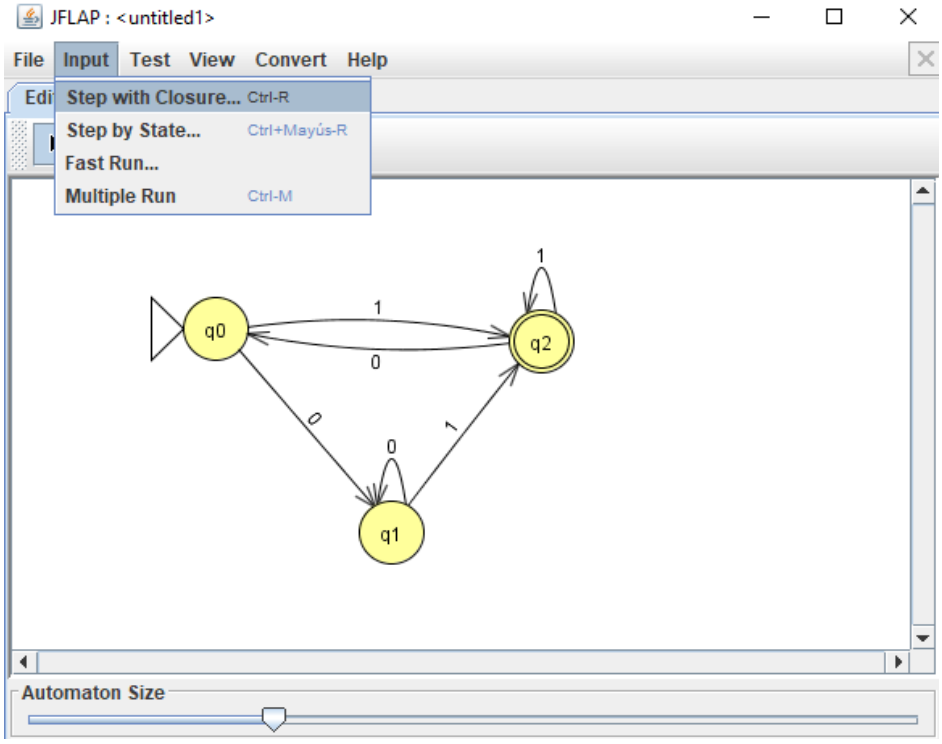
8. Realizamos el mismo proceso con el estado q_1 y se le asigna a la transición un valor de 0:



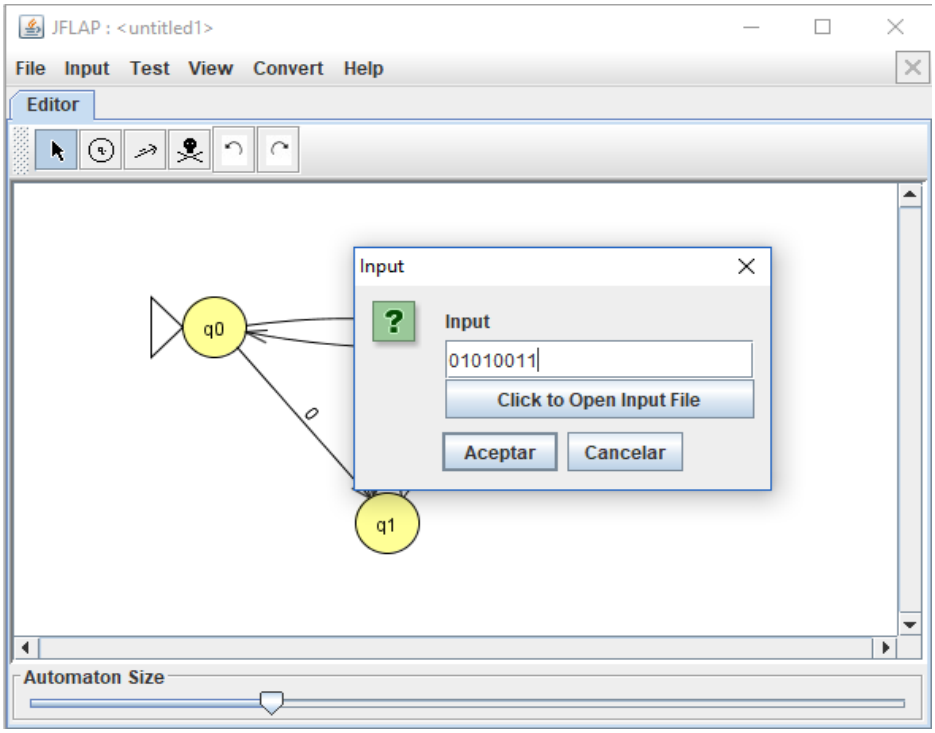
9. Para terminar, volvemos a seleccionar la 1 opción de izquierda a derecha del editor para indicar al software nuestro estado final:



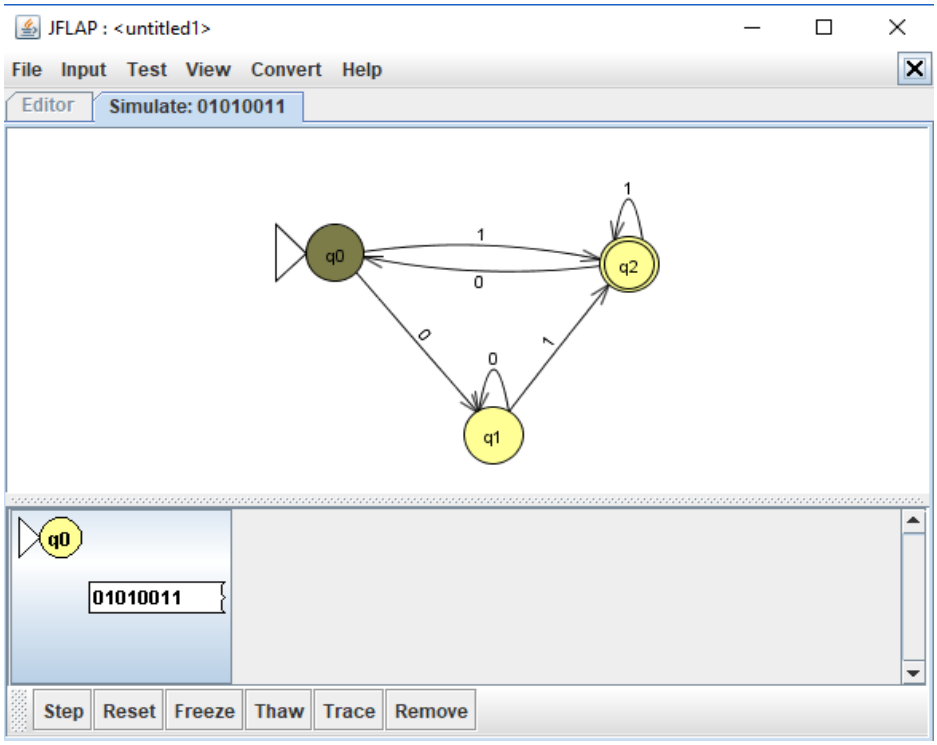
10. Para probar nuestro autómata con una cadena x seguimos los siguientes pasos $Input - Step\ with\ Closure$:

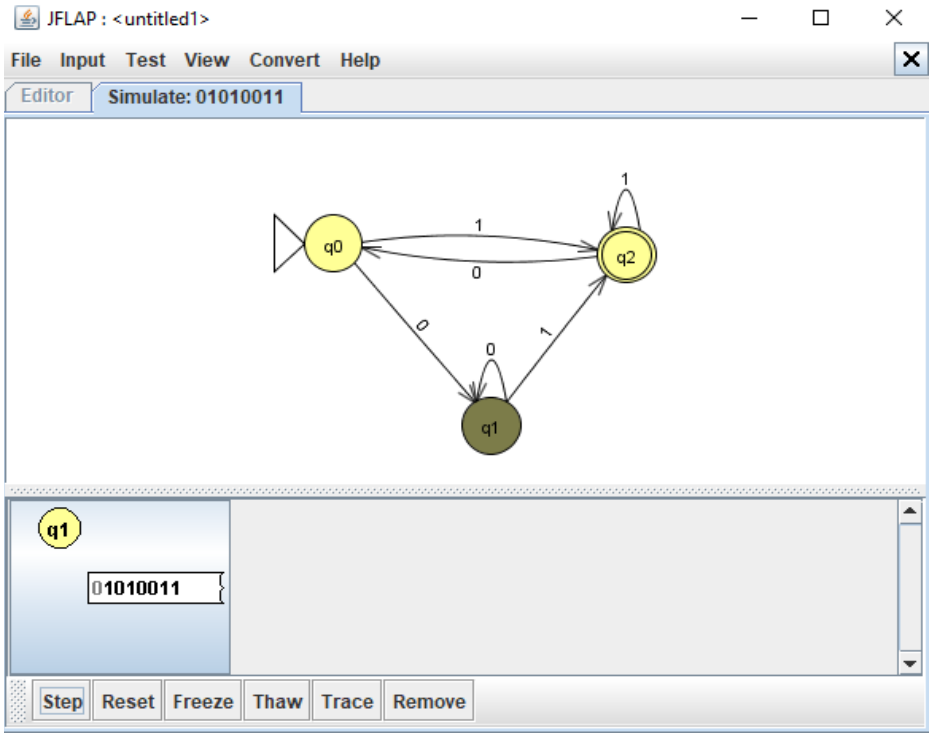


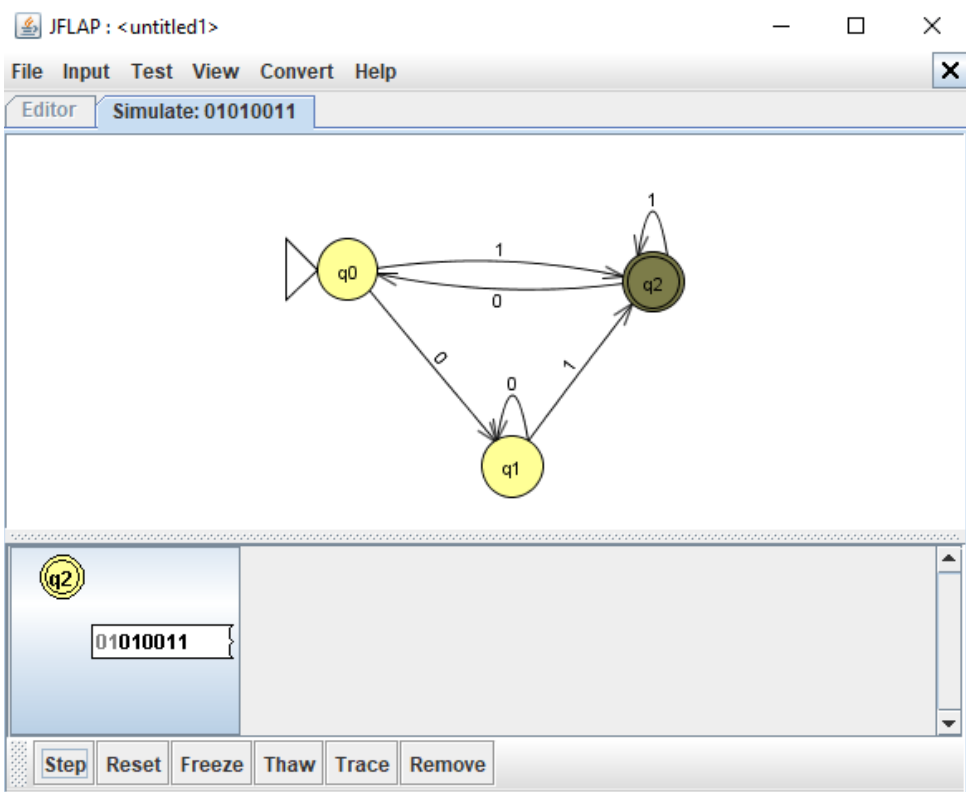
11. Ingresamos nuestra cadena con la cual probaremos nuestro autómata en nuestro caso será = 01010011

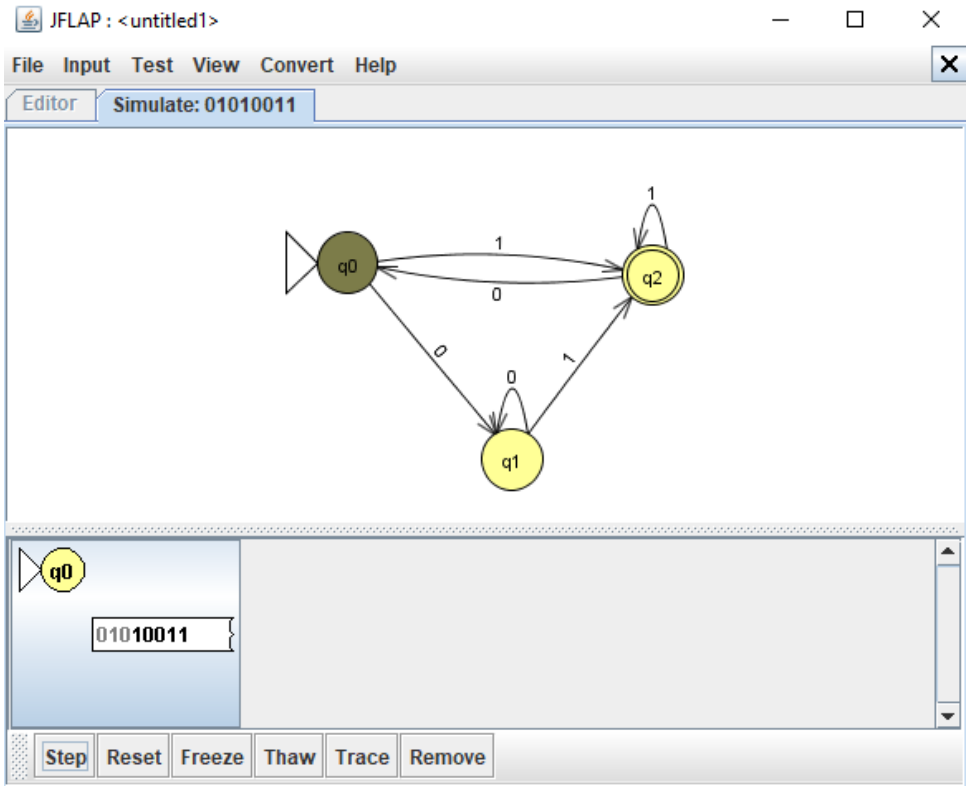


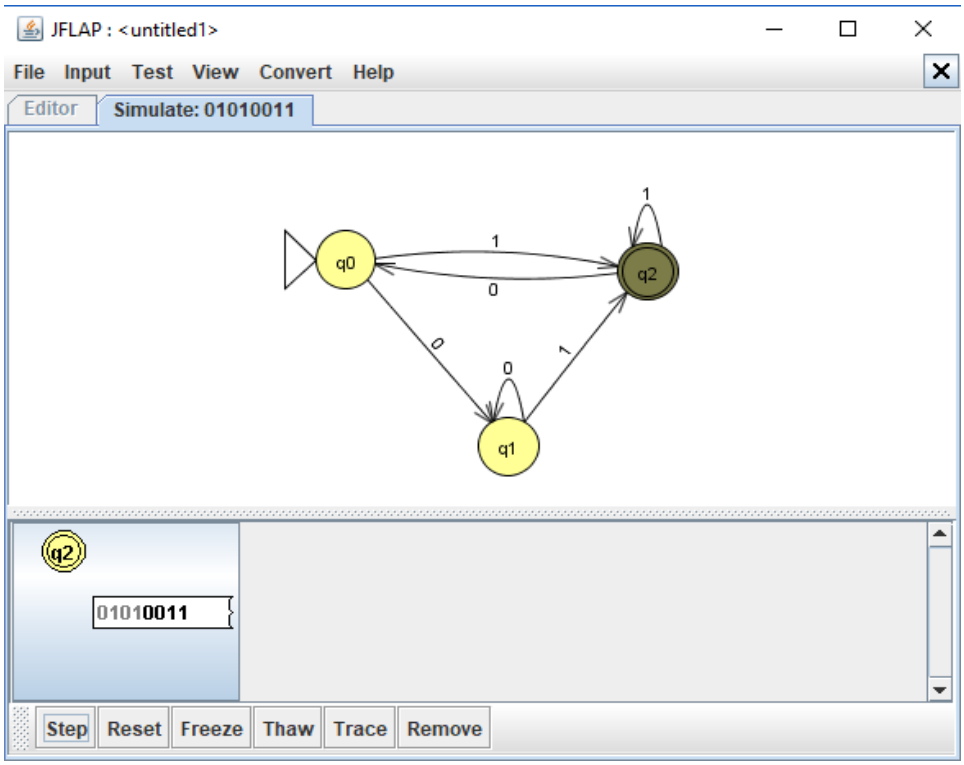
12. Luego para ver los pasos de la simulación – recorrido que realiza la cadena por el autómata pulsamos la opción *Step*:

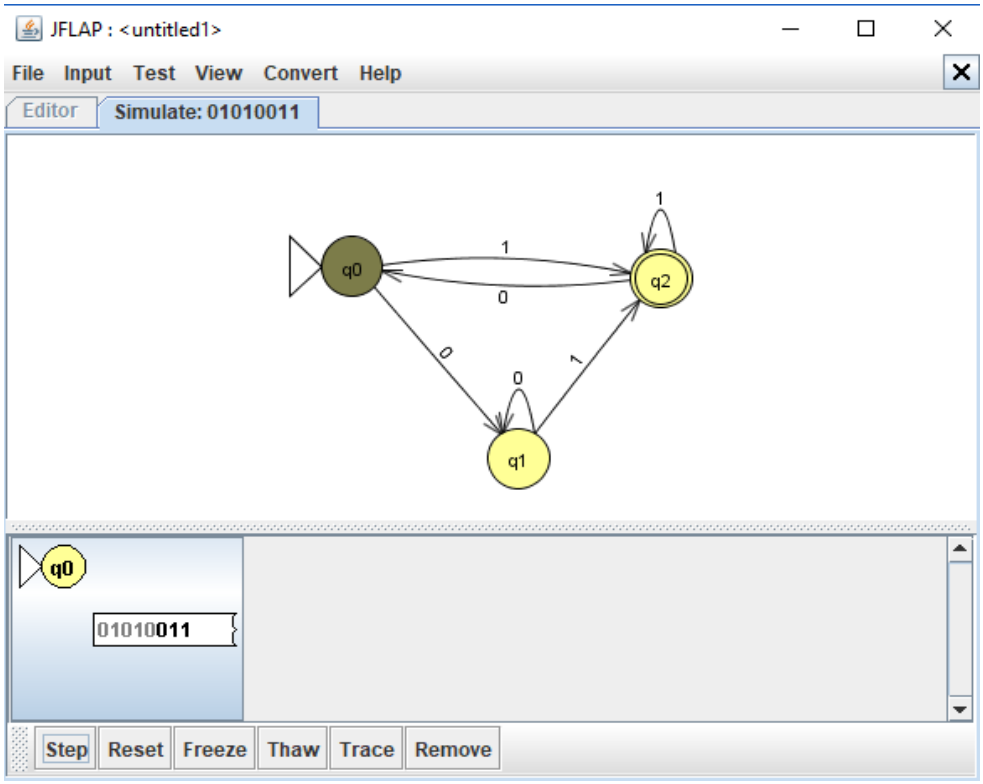


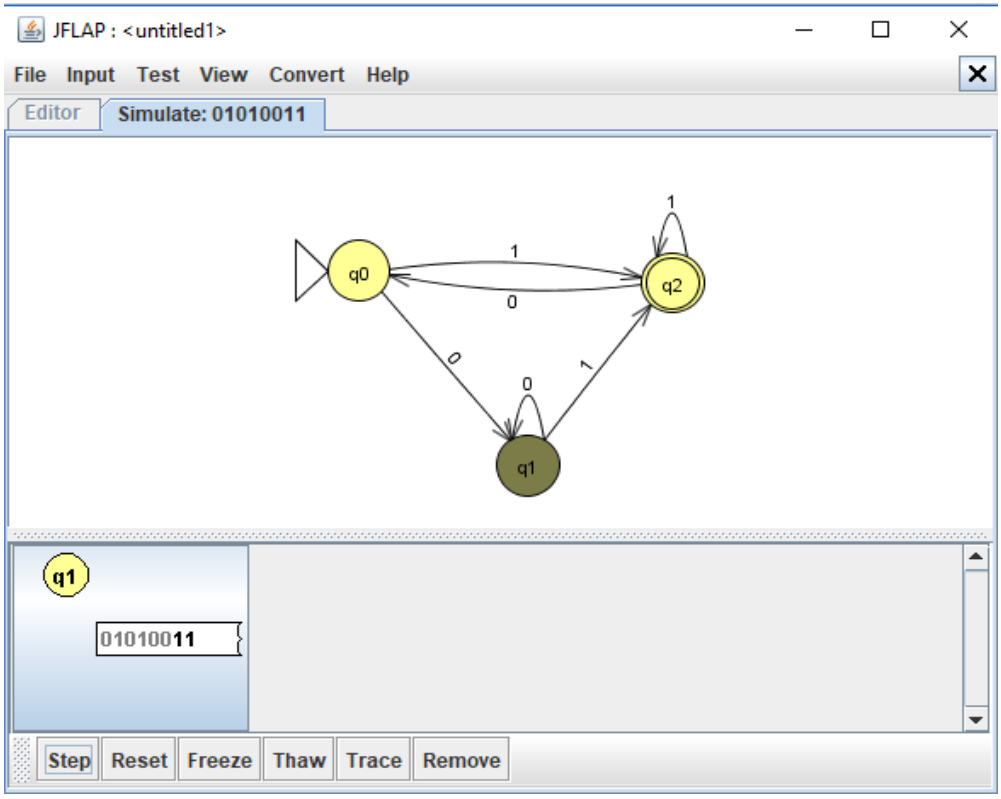

JFLAP : <untitled1>
File Input Test View Convert Help
Editor
Simulate: 01010011
q0
q1
q2
1
0
1
0
0
1
q1
01010011
Step Reset Freeze Thaw Trace Remove

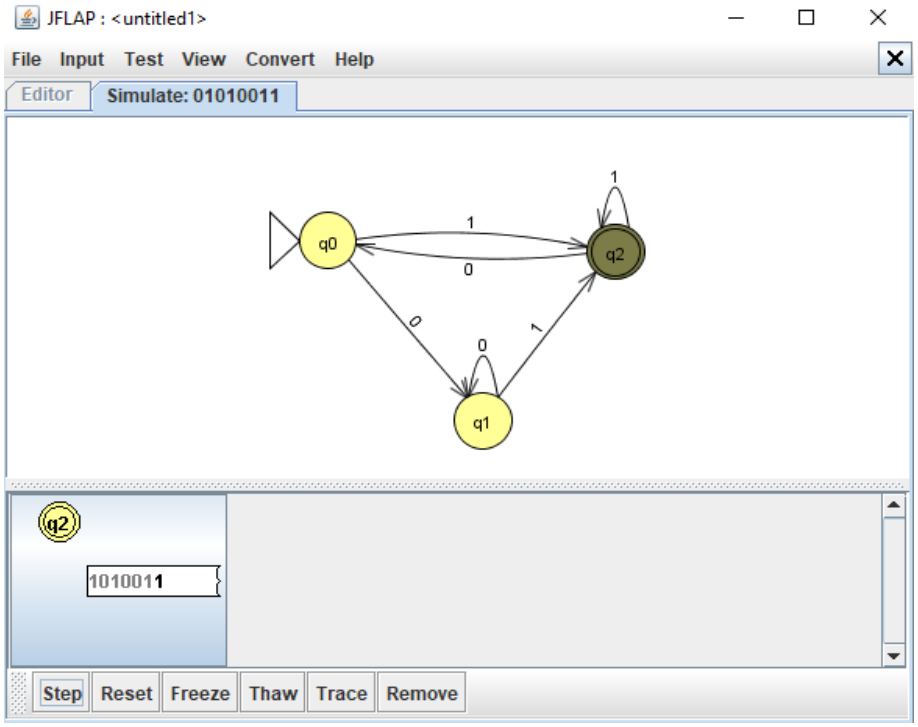

JFLAP : <untitled1>
File Input Test View Convert Help
Editor
Simulate: 01010011
q0
q1
q2
1
0
1
0
0
1
q2
01010011
Step Reset Freeze Thaw Trace Remove


JFLAP : <untitled1>
File Input Test View Convert Help
Editor
Simulate: 01010011
q0
q1
q2
1
0
01010011
Step Reset Freeze Thaw Trace Remove

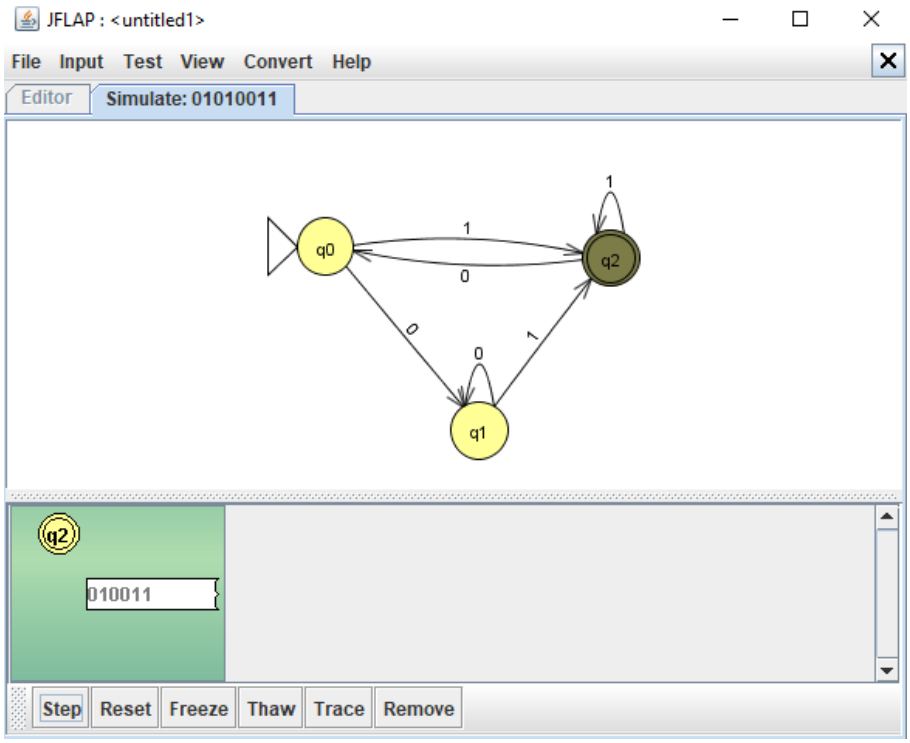

JFLAP : <untitled1>
File Input Test View Convert Help
Editor
Simulate: 01010011
q0
q1
q2
1
0
01010011
Step Reset Freeze Thaw Trace Remove


JFLAP : <untitled1>
File Input Test View Convert Help
Editor
Simulate: 01010011
q0
q1
q2
1
0
01010011
Step Reset Freeze Thaw Trace Remove


JFLAP : <untitled1>
File Input Test View Convert Help
Editor
Simulate: 01010011
q0
q1
q2
1
0
01010011
Step Reset Freeze Thaw Trace Remove



13. Efectivamente esta cadena es aceptada por este autómata la cual su condición a cumplir era:



Referencias

- *cs.duke*. (s.f.). Obtenido de http://www.cs.duke.edu/csed/jflap/
- *jflap*. (s.f.). Obtenido de http://www.jflap.org/